DISCRETE STRUCTURES

Mid-Term Study Guide



dawaralibhatti@gmail.com

Contents

1.1 Logical form and logical equivalence 5

Statements (propositions) 5
The connectives 5
Inclusive vs exclusive “or” 7
Translating English to symbols 8
Order of operations 9
How to build a truth table 9
Logical equivalence 10
Tautologies and contradictions 12
The laws of logic (memorise this table) 13
Simplifying with the laws 13

1.2 Conditional statements 16

Definition 16
Implication law and its consequences 18
Negation of a conditional 18
Contrapositive, converse and inverse 19
Only if, biconditional 20
Necessary and sufficient conditions 20
Worked truth tables 23

1.3 Valid and invalid arguments 24

Definitions 24
Testing validity: the critical-row method 24
Rules of inference 26
Fallacies (invalid forms that look valid) 27

dawaralibhatti@gmail.com

1.4 Application: digital logic circuits 29

Gates 29

Three conversions you must be able to do 30

Simplifying a circuit 32

NAND and NOR (universal gates) 32

1.5 Application: number systems and circuits for addition 34

Binary representation 34

Binary addition and subtraction 35

Adder circuits 36

Two’s complement and negative integers (8-bit) 37

Hexadecimal 39

2.1 Predicates and quantified statements I 41

Predicates and domains 41

The universal quantifier ∀ 41

The existential quantifier ∃ 42

Universal conditional statements 43

Negating quantified statements 43

Equivalent forms 44

Bound variables, scope, implicit quantification 45

Tarski’s world 45

Quick revision 46

Ten things to write from memory 46

Common mistakes 46

Self-test 47

dawaralibhatti@gmail.com

The Exam Map

Where each topic lives in the course and what the exam usually asks you to do.

Sec.	Topic	Typical question
1.1	Statements, connectives, truth tables, equivalence, laws	• Build a truth table • Show two statements are equivalent • Simplify with laws • De Morgan negation
1.2	Conditionals	• Truth table of p→q • Negate a conditional • Write converse / inverse / contrapositive • “Only if”, necessary / sufficient
1.3	Valid and invalid arguments	• Decide validity with a truth table or a rule of inference • Name the fallacy
1.4	Digital logic circuits	• Circuit ↔ Boolean expression ↔ input/output table • Simplify • NAND / NOR
1.5	Binary, two’s complement, hex	• Convert, add, represent negatives in 8 bits
2.1	Predicates and quantifiers	• Truth value of ∀ / ∃ statements • Negate them • Translate English

HOW TO USE THIS GUIDE

Every section below ends its topic with the same habit: **show your steps**. In truth tables and law-by-law simplifications, the marks are in the working, not just the final answer.

dawaralibhatti@gmail.com

1.1 Logical form and logical equivalence

Statements (propositions)

KEY IDEA

A **statement** is a declarative sentence that is **true or false, but not both**.

Sentence	Statement?	Why
Paris is the capital of France.	**Yes (T)**	Declarative, has a truth value
Can Ali come with you?	**No**	Question
Take two aspirins.	**No**	Command
$x + 4 > 9$	**No**	True for $x = 6$, false for $x = 5$, so no fixed truth value
He is a college student.	**No**	"He" is not identified

Propositional variables are lowercase letters **p, q, r, s**. Truth values are **T** (or 1) and **F** (or 0).

The connectives

A **compound statement** is built from simpler statements with connectives. A **truth table** lists every combination of truth values of the inputs. With **n variables there are 2^n rows**.

dawaralibhatti@gmail.com

p	q	~p	p∧q	p∨q	p⊕q	p→q	p↔q	p↑q NAND	p↓q NOR
T	T	F	T	T	F	T	T	F	F
T	F	F	F	T	T	F	F	T	F
F	T	T	F	T	T	T	F	T	F
F	F	T	F	F	F	T	T	T	T

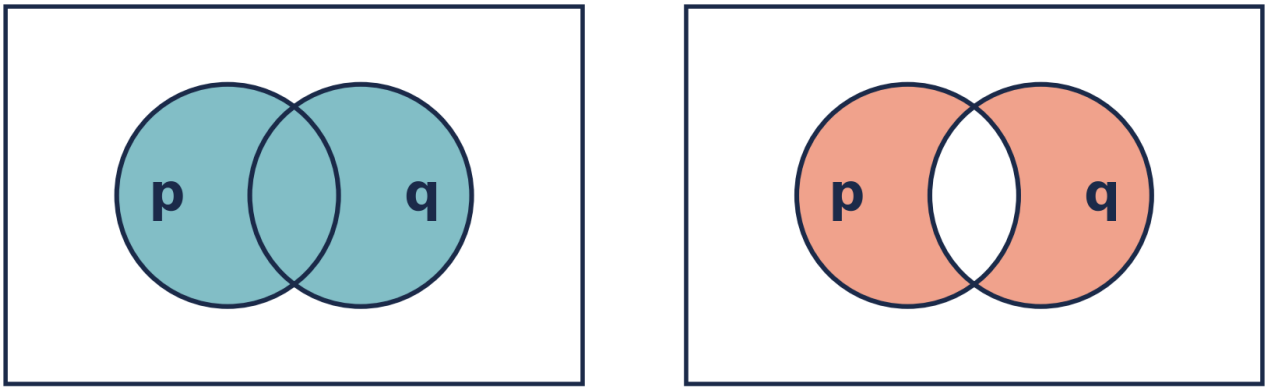
One-line memory aids

- **AND** is true only when **both** are true.
- **OR** is false only when **both** are false.
- **XOR** is true when the two **differ**. It is the opposite of ↔.
- **p→q** is false only when p is true and q is false.
- **p↔q** is true when p and q **match**.
- **NAND** = NOT(AND). **NOR** = NOT(OR).

> **REMEMBER**
> The three fundamental connectives are **NOT, AND and OR**. Everything else can be built from them.

dawaralibhatti@gmail.com

Inclusive vs exclusive “or”



Inclusive $p \vee q$ **Exclusive $p \oplus q$**

Inclusive OR shades everything in p or q (overlap included). Exclusive OR leaves the overlap out.

- **Inclusive ($\vee$):** at least one is true. “To register you must have passed the qualifying exam or be a Math major.” Having both is fine.
- **Exclusive ($\oplus$):** exactly one is true. “Nabeel has one or two brothers.” He cannot have both.

EXCLUSIVE OR IN BASIC CONNECTIVES

$p \oplus q \equiv (p \vee q) \wedge \sim(p \wedge q)$

dawaralibhatti@gmail.com

Translating English to symbols

English	Symbols
p and q; p but q; p, although q	$p \wedge q$
p or q (or both)	$p \vee q$
not p; it is not the case that p	$\sim p$
either p or q, but not both	$p \oplus q$
neither p nor q	$\sim p \wedge \sim q$
if p then q	$p \rightarrow q$
p if and only if q	$p \leftrightarrow q$

Example

“I did not buy a lottery ticket this week **or** I bought a lottery ticket **and** won the million on Friday.”

Let **p** = I bought a ticket this week, **q** = I won on Friday. Then the statement is $\sim p \vee (p \wedge q)$.

dawaralibhatti@gmail.com

Order of operations



1st	2nd	3rd	4th	last
Example	p ∨ ~q → ~p	means	(p ∨ (~q)) → (~p)	

Exam habit: add your own brackets so nobody (including you) has to guess.

- Brackets first, then ~ (NOT), then ∧ and ∨, then →, then ↔ last.
- Your Lecture 1 slides list the order as ( ), NOT, AND, OR, →, ↔. Epp's textbook gives ∧ and ∨ **equal rank**, so a mixed ∧ / ∨ expression without brackets is ambiguous.

> **EXAM TIP**
> Put brackets in your own answer and follow your instructor's convention. For example, **p∨~q→~p** means **(p∨(~q))→(~p)**.

How to build a truth table

1. List every variable and write all **2^n** combinations. Standard order is TTT, TTF, TFT, TFF, FTT, FTF, FFT, FFF.
2. Add **one column per sub-expression**, working from the innermost brackets outward.
3. The last column is the answer. **Never skip a column; the marks are in the steps.**

dawaralibhatti@gmail.com

Example: ~p ∧ (q ∨ ~r)

p	q	r	~p	~r	q∨~r	~p∧(q∨~r)
T	T	T	F	F	T	F
T	T	F	F	T	T	F
T	F	T	F	F	F	F
T	F	F	F	T	T	F
F	T	T	T	F	T	T
F	T	F	T	T	T	T
F	F	T	T	F	F	F
F	F	F	T	T	T	T

Colour key: **navy** = inputs, **teal** = working columns, **orange** = the final answer.

Logical equivalence

KEY IDEA

Two statement forms are **logically equivalent** (written ≡) if they have the **same truth value for every** combination of truth values of their variables.

- **To prove equivalence:** build a truth table for both and check that the two final columns match in every row.
- **To disprove equivalence:** show one row where they differ.
- **Double negation:** ~(~p) ≡ p.

dawaralibhatti@gmail.com

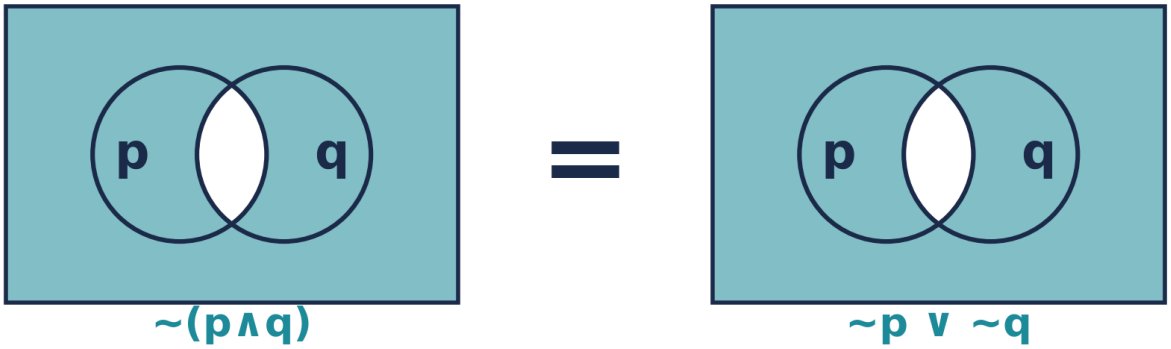
De Morgan’s laws (the most-tested pair)

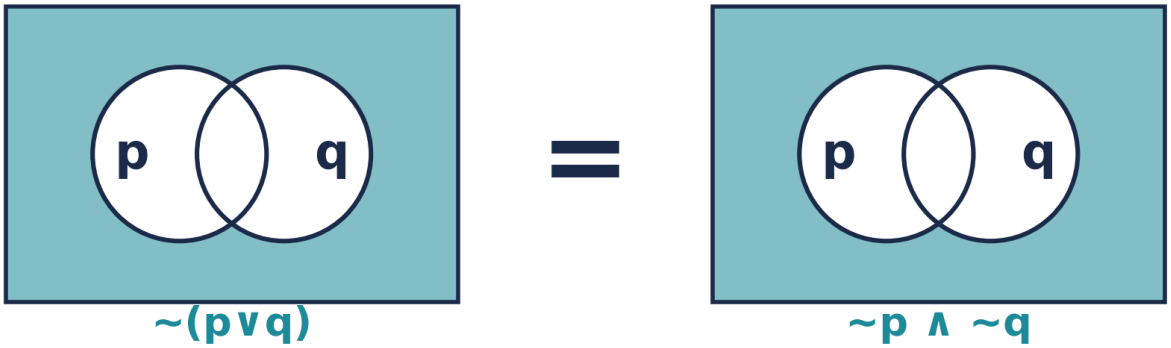
DE MORGAN

$\sim(p \wedge q) \equiv \sim p \vee \sim q$

$\sim(p \vee q) \equiv \sim p \wedge \sim q$

In words: negating **flips AND ↔ OR** and **negates each part**.





Shaded regions are “true”. Each pair of pictures is identical, which is exactly what ≡ means.

WATCH OUT

$\sim(p \wedge q)$ is **NOT** $\sim p \wedge \sim q$. This is the most common mistake.

dawaralibhatti@gmail.com

De Morgan negations in English (practice sheet Q3)

Statement	Negation
Hal is a Math major **and** Hal's sister is a Computer Science major.	Hal is not a Math major **or** Hal's sister is not a Computer Science major.
Sam is orange belt **and** Kate is red belt.	Sam is not orange belt **or** Kate is not red belt.
The connector is loose **or** the machine is unplugged.	The connector is not loose **and** the machine is not unplugged.

Tautologies and contradictions

Type	Meaning	Example
Tautology (t)	True in every row	**p ∨ ~p**
Contradiction (c)	False in every row	**p ∧ ~p**
Contingency	Sometimes true, sometimes false	**p ∧ q**

Practice sheet Q1(a)

~(p∧q) ∨ (p∨q) is a tautology. Algebraic proof:

```
(~p∨~q)∨(p∨q)
≡ (p∨~p)∨(q∨~q)
≡ t ∨ t
≡ t
```

dawaralibhatti@gmail.com

The laws of logic (memorise this table)

Law	Form 1	Form 2
Commutative	p∧q ≡ q∧p	p∨q ≡ q∨p
Associative	(p∧q)∧r ≡ p∧(q∧r)	(p∨q)∨r ≡ p∨(q∨r)
Distributive	p∧(q∨r) ≡ (p∧q)∨(p∧r)	p∨(q∧r) ≡ (p∨q)∧(p∨r)
Identity	p∧t ≡ p	p∨c ≡ p
Negation	p∨~p ≡ t	p∧~p ≡ c
Double negation	~(~p) ≡ p	
Idempotent	p∧p ≡ p	p∨p ≡ p
Universal bound (domination)	p∨t ≡ t	p∧c ≡ c
De Morgan	~(p∧q) ≡ ~p∨~q	~(p∨q) ≡ ~p∧~q
Absorption	p∨(p∧q) ≡ p	p∧(p∨q) ≡ p
Negations of t and c	~t ≡ c	~c ≡ t

Note: **F** plays the role of **c** and **T** plays the role of **t**.

Simplifying with the laws

Write each step on its own line with the **law named on the right**. Marks are for the law names.

dawaralibhatti@gmail.com

Example 1: simplify p ∨ [~(~p ∧ q)]

Step	Law
p ∨ ~(~p∧q)	*given*
≡ p ∨ (~(~p) ∨ ~q)	*De Morgan*
≡ p ∨ (p ∨ ~q)	*Double negation*
≡ (p ∨ p) ∨ ~q	*Associative*
≡ p ∨ ~q	*Idempotent*

Example 2: verify ~(~p ∧ q) ∧ (p ∨ q) ≡ p

Step	Law
~(~p∧q) ∧ (p∨q)	*given*
≡ (~(~p) ∨ ~q) ∧ (p∨q)	*De Morgan*
≡ (p ∨ ~q) ∧ (p ∨ q)	*Double negation*
≡ p ∨ (~q ∧ q)	*Distributive (in reverse)*
≡ p ∨ c	*Negation*
≡ p	*Identity*

Example 3 (translation + distributive in reverse)

"You get an A if you are hardworking and the sun shines, **or** you are hardworking and it rains."

With **p** = hardworking, **q** = sun shines, **r** = rains, this is **`(p∧q)∨(p∧r) ≡ p∧(q∨r)`**.

In English: you get an A if you are hardworking **and** the sun shines **or** it rains.

dawaralibhatti@gmail.com

Example 4

- Is **p∨(p∧q) ≡ p**? **Yes.** The two columns match (Absorption).
- Is **p∧c ≡ p∨c**? **No.** p∧c ≡ c but p∨c ≡ p, and they differ when p is true.

dawaralibhatti@gmail.com

1.2 Conditional statements

Definition

KEY IDEA

If p and q are statements, “if p then q” is written **p→q**. **p** is the **hypothesis** (antecedent). **q** is the **conclusion** (consequent).

p	q	p→q
T	T	T
T	F	F
F	T	T
F	F	T

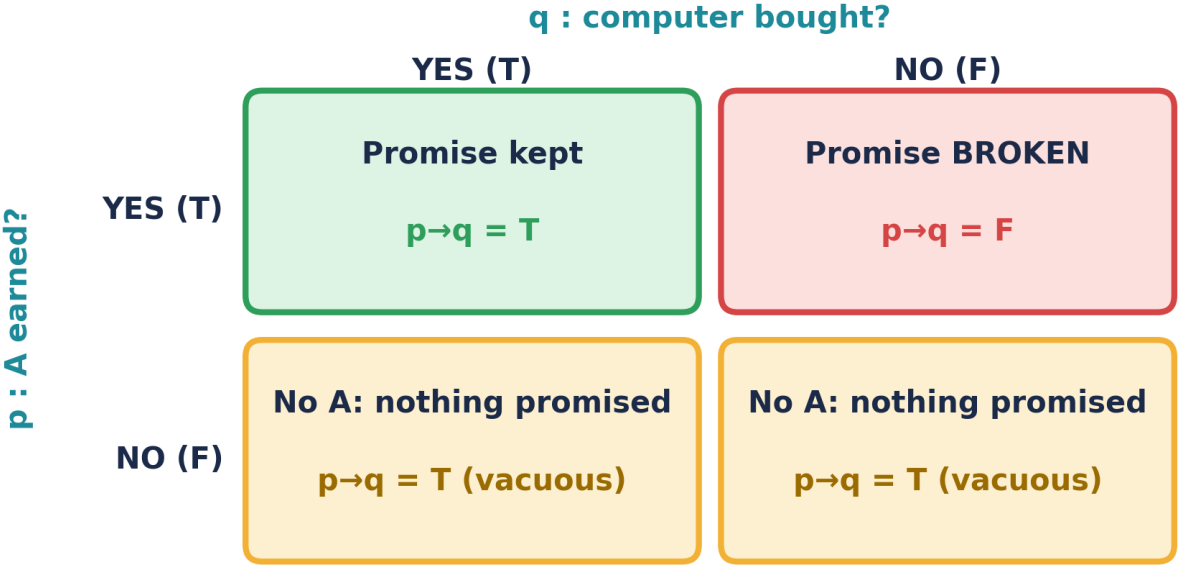
- **p→q is false only when p is true and q is false.** In every other row it is true.
- If the hypothesis is false the conditional is true automatically. This is called **vacuously true** or true by default.

dawaralibhatti@gmail.com

Think of it as a promise

"If you earn an A in Math, I'll buy you a computer."

p = you earn an A q = I buy the computer



The only way the promise is broken: you DO earn the A and I do NOT buy the computer.

Slide examples

Statement	Truth value
If 1 = 1, then 3 = 3.	**True**
If 1 = 1, then 2 = 3.	**False**
If 1 = 0, then 3 = 3.	**True**
If 1 = 2, then 2 = 3.	**True**

dawaralibhatti@gmail.com

Implication law and its consequences

IMPLICATION LAW

$p \to q \equiv \sim p \vee q$

Use this to eliminate → so that the other laws apply.

Check the table: ~p∨q is false only when p is true and q is false.

Negation of a conditional

NEGATION OF A CONDITIONAL

$\sim(p \to q) \equiv p \wedge \sim q$

Proof: ~(p→q) ≡ ~(~p∨q) ≡ ~(~p) ∧ ~q ≡ p ∧ ~q *(implication law, De Morgan, double negation)*.

WATCH OUT

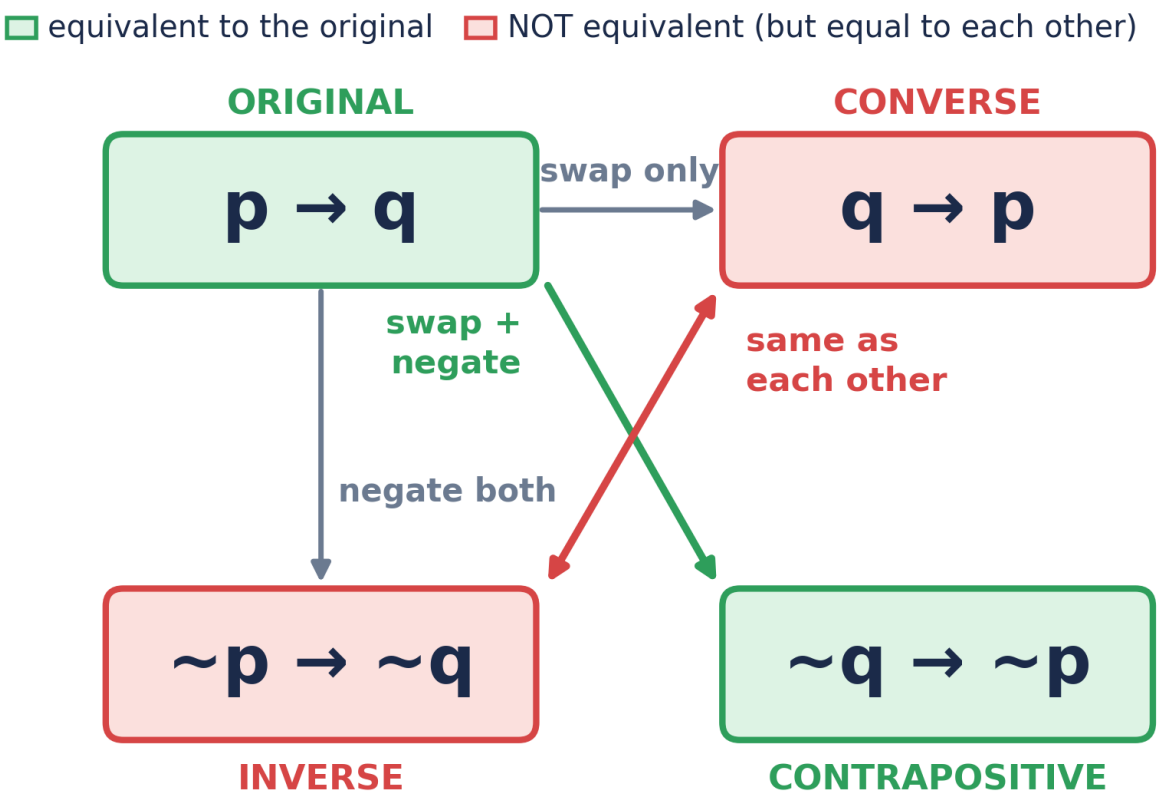
The negation of an if-then is an **AND** statement, not another if-then.

Conditional	Negation
If Ali lives in Pakistan then he lives in Lahore.	Ali lives in Pakistan **and** he does not live in Lahore.
If my car is in the repair shop, then I cannot get to class.	My car is in the repair shop **and** I can get to class.
If x is prime then x is odd or x = 2.	x is prime **but** x is not odd **and** x is not 2.
If n is divisible by 6, then n is divisible by 2 and n is divisible by 3.	n is divisible by 6 **but** n is not divisible by 2 **or** not divisible by 3.

dawaralibhatti@gmail.com

Contrapositive, converse and inverse

Start from **p → q**.



Name	Form	How to get it	Equivalent to p→q?
Contrapositive	`~q → ~p`	Swap and negate both	**Yes, always**
Converse	`q → p`	Swap only	**No**
Inverse	`~p → ~q`	Negate both only	**No**

- The converse and the inverse are equivalent to **each other** (each is the contrapositive of the other), but not to the original.
- Proof that p→q ≡ ~q→~p: both columns read **T, F, T, T**.

dawaralibhatti@gmail.com

Example

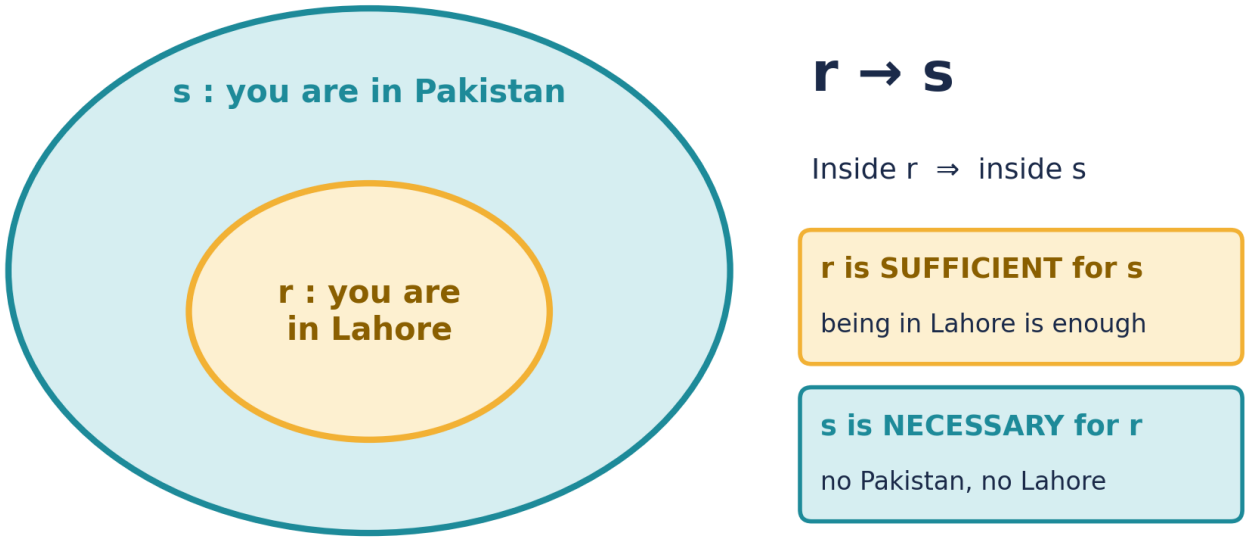
Original: “If it is noon, then I am hungry.”

- **Converse:** If I am hungry, then it is noon.
- **Inverse:** If it is not noon, then I am not hungry.
- **Contrapositive:** If I am not hungry, then it is not noon.

Only if, biconditional

- “p **only if** q” means **p → q**. It does **NOT** mean q → p.
- **p ↔ q** means (p→q) ∧ (q→p). Read as “p if and only if q” (iff).
- p ↔ q is true when p and q have the **same truth value**.
- p ↔ q ≡ ~(p ⊕ q).
- Hierarchy again: ↔ is evaluated **last**, so p∧q ↔ r means (p∧q) ↔ r.

Necessary and sufficient conditions



- “r is a **sufficient** condition for s” means **r → s**.

dawaralibhatti@gmail.com

- “r is a **necessary** condition for s” means ~r → ~s, which is equivalent to **s → r**.
- “r is necessary and sufficient for s” means **r ↔ s**.

All the ways of writing p → q

- → if p then q; p implies q; if p, q
- → p only if q
- → p is sufficient for q
- → q follows from p; q if p; q whenever p
- → q is necessary for p
- → not p unless q

Examples

English	If-then form
Your guarantee is good **only if** you bought your CD less than 90 days ago.	If your guarantee is good, then you bought it less than 90 days ago.
To get tenure it is **sufficient** to be world-famous.	If you are world-famous, then you get tenure.
That you get the job **implies** you have the best credentials.	If you get the job, then you have the best credentials.
It is **necessary** to walk 8 miles to get to the top.	If you get to the top, then you must have walked 8 miles.

Translation with letters

Let **p** = you get an A on the final, **q** = you do every exercise in the book, **r** = you get an A in the class.

dawaralibhatti@gmail.com

English	Symbols
To get an A in this class it is **necessary** to get an A on the final.	`r → p`
You do every exercise and get an A on the final **implies** you get an A in the class.	`p∧q → r`
Getting an A on the final and doing every exercise is **sufficient** for getting an A in the class.	`p∧q → r`

Symbols to English

With **p** = flu, **q** = miss the final, **r** = pass the course:

`~p∧~q → r` reads “If you neither have the flu nor miss the final exam, then you will pass the course.”

dawaralibhatti@gmail.com

Worked truth tables

(p→q) ∧ (~p→r)

p	q	r	p→q	~p	~p→r	result
T	T	T	T	F	T	T
T	T	F	T	F	T	T
T	F	T	F	F	T	F
T	F	F	F	F	T	F
F	T	T	T	T	T	T
F	T	F	T	T	F	F
F	F	T	T	T	T	T
F	F	F	T	T	F	F

EXAM TIP

Shortcut: when **p is true** the result equals **q**; when **p is false** it equals **r**.

p ∨ ~q → ~p [means (p ∨ ~q) → (~p)]

p	q	~q	p∨~q	~p	(p∨~q)→~p
T	T	F	T	F	F
T	F	T	T	F	F
F	T	F	F	T	T
F	F	T	T	T	T

dawaralibhatti@gmail.com

1.3 Valid and invalid arguments

Definitions

- An **argument** is a list of statements called **premises** (hypotheses) followed by a statement called the **conclusion**. Written with ∴ before the conclusion.
- An argument form is **valid** if, whenever **all the premises are true**, the conclusion is also true. Validity is about the **form**, not about whether the statements are true in real life.
- A valid argument with true premises is called **sound**. A valid argument can still have a false conclusion if a premise is false.
- An argument form is **invalid** if there is some assignment where all premises are true but the conclusion is false.

Testing validity: the critical-row method



dawaralibhatti@gmail.com

Example: p → q, q ⊢ p (the converse error)

p	q	p→q (premise)	q (premise)	p (conclusion)
T	T	T	T	T
T	F	F	F	T
F	T	T	T	F
F	F	T	F	F

Critical rows (outlined in gold) are the first and third. In the third row the premises are true but the conclusion is false, so the argument is **invalid**.

dawaralibhatti@gmail.com

Rules of inference

Rule	Form	Example
Modus ponens	p→q; p; ∴ q	If it rains, the road is wet. It rains. ∴ The road is wet.
Modus tollens	p→q; ~q; ∴ ~p	If it rains, the road is wet. The road is not wet. ∴ It did not rain.
Generalization	p; ∴ p∨q (and q; ∴ p∨q)	I am at home. ∴ I am at home or at work.
Specialization	p∧q; ∴ p (and ∴ q)	It is cold and windy. ∴ It is cold.
Conjunction	p; q; ∴ p∧q	
Elimination	p∨q; ~q; ∴ p (and p∨q; ~p; ∴ q)	The light is on or the power is out. The power is not out. ∴ The light is on.
Transitivity (hypothetical syllogism)	p→q; q→r; ∴ p→r	
Proof by division into cases	p∨q; p→r; q→r; ∴ r	
Contradiction rule	~p→c; ∴ p	If assuming ~p leads to a contradiction, p is true.

dawaralibhatti@gmail.com

Fallacies (invalid forms that look valid)

✓ MODUS PONENS

$p \to q$
p
$\therefore q$

VALID
affirm the hypothesis

rain → wet road; it rains;
so the road is wet

✓ MODUS TOLLENS

$p \to q$
$\neg q$
$\therefore \neg p$

VALID
deny the conclusion

rain → wet road; road is dry;
so it did not rain

✗ CONVERSE ERROR

$p \to q$
q
$\therefore p$

INVALID
affirm the conclusion

road is wet; so it rained?
(a garden hose, maybe)

✗ INVERSE ERROR

$p \to q$
$\neg p$
$\therefore \neg q$

INVALID
deny the hypothesis

no rain; so the road is dry?
(a garden hose, maybe)

- **Converse error:** p→q; q; ∴ p. (Concluding the hypothesis from the conclusion.)
- **Inverse error:** p→q; ~p; ∴ ~q. (Denying the hypothesis.)
- Both are **invalid**. Modus ponens affirms the hypothesis, and modus tollens denies the conclusion. These are the **only two safe moves**.

Example of the inverse error

"If the team wins, they celebrate. The team did not win. ∴ They did not celebrate." **Invalid**, since they might celebrate for another reason.

dawaralibhatti@gmail.com

Example of a valid chain

Premises: **p→q, q→r, ~r**.

1. Modus tollens on q→r gives **~q**.
2. Modus tollens on p→q gives **~p**.
3. Conclusion: **~p**.

Syllogism, deductive and inductive reasoning

All humans are mortal; Socrates is human; ∴ Socrates is mortal.

- This is **deductive** reasoning: the conclusion is guaranteed if the premises hold.
- **Inductive** reasoning (the sun has risen every day, so it will rise tomorrow) is only probable.

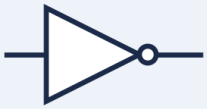
dawaralibhatti@gmail.com

1.4 Application: digital logic circuits

Gates

A digital circuit takes inputs of **1** (true / high voltage) and **0** (false / low voltage) and produces an output. Each gate matches a logical connective.

NOT



~P

output is 1 when the input is 0

AND



P∧Q

output is 1 when both inputs are 1

OR



P∨Q

output is 1 when at least one input is 1

XOR



P⊕Q

output is 1 when the inputs differ

NAND



P↑Q = ~(P∧Q)

output is 1 unless both inputs are 1

NOR



P↓Q = ~(P∨Q)

output is 1 only when both are 0

dawaralibhatti@gmail.com

Gate	Symbol in logic	Output is 1 when
NOT (inverter)	**~P**	the input is 0
AND	**P∧Q**	both inputs are 1
OR	**P∨Q**	at least one input is 1
XOR	**P⊕Q**	the inputs differ
NAND	**P↑Q = ~(P∧Q)**	not both inputs are 1
NOR	**P↓Q = ~(P∨Q)**	both inputs are 0

Three conversions you must be able to do

1. **Circuit → Boolean expression:** label the output of every gate from the inputs forward and read the expression off. Example: P and Q go into an AND, the result goes into an OR with R. The expression is **(P∧Q)∨R**.
2. **Boolean expression → circuit:** start with the **outermost operator**, which is the last gate, then work inward. Order of operations decides the gate order.
3. **Input/output table → circuit (sum of products):** see below.

Sum of products, step by step

- → Pick every row where the output is **1**.
- → For each such row write an **AND** of all inputs, using ~ on every input that is 0 in that row.
- → **OR** all those AND terms together.
- → Simplify with the laws.

dawaralibhatti@gmail.com

Example: S is 1 only in the rows PQR = 111 and 101

P	Q	R	S
1	1	1	1
1	1	0	0
1	0	1	1
1	0	0	0
0	1	1	0
0	1	0	0
0	0	1	0
0	0	0	0

1 Rows with S=1

PQR = 111
and 101

2 AND each row

P∧Q∧R
P∧~Q∧R

3 OR the terms

(P∧Q∧R) ∨
(P∧~Q∧R)

4 Simplify

(P∧R)∧(Q∨~Q)
= (P∧R)∧t = P∧R

Result: the whole circuit collapses to ONE AND gate



Q is ignored

dawaralibhatti@gmail.com

Simplifying a circuit

A circuit with **fewer gates** is cheaper and faster. Convert it to an expression, simplify with the laws from section 1.1, then redraw it. Two circuits are equivalent if their expressions are logically equivalent.

NAND and NOR (universal gates)

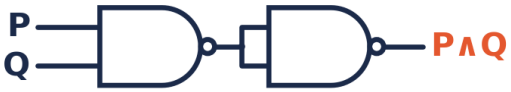
- **Sheffer stroke:** P ↑ Q ≡ ~(P∧Q). **Peirce arrow:** P ↓ Q ≡ ~(P∨Q).
- Either gate **alone** can build any circuit, which is why they are called **universal**.

~P = P ↑ P



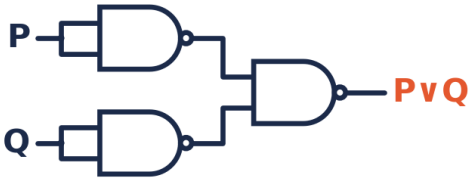
tie both inputs together

P∧Q = (P↑Q) ↑ (P↑Q)



P∨Q = (P↑P) ↑ (Q↑Q)

NOR does the same job:
P↓P = ~P
(P↓P)↓(Q↓Q) = P∧Q
(P↓Q)↓(P↓Q) = P∨Q



Building NOT, AND and OR from NAND gates only.

To get	Using NAND only	Using NOR only
~P	P ↑ P	P ↓ P
P ∧ Q	(P↑Q) ↑ (P↑Q)	(P↓P) ↓ (Q↓Q)
P ∨ Q	(P↑P) ↑ (Q↑Q)	(P↓Q) ↓ (P↓Q)

dawaralibhatti@gmail.com

WHY IT WORKS

Why P↑P ≡ ~P: P↑P = ~(P∧P) = ~P by idempotence.

Why (P↑Q)↑(P↑Q) ≡ P∧Q: it is the negation of ~(P∧Q), so double negation gives P∧Q.

dawaralibhatti@gmail.com

1.5 Application: number systems and circuits for addition

Binary representation

Place values are powers of 2: … 128, 64, 32, 16, 8, 4, 2, 1.



110101 = 32 + 16 + 4 + 1 = 53

Binary → decimal

Add the place values where the digit is 1. 110101 = 32 + 16 + 4 + 1 = **53**.

Decimal → binary

Divide by 2 repeatedly and read the remainders from **bottom to top**. Example: 53.

dawaralibhatti@gmail.com

Division	Quotient	Remainder
53 ÷ 2	26	1
26 ÷ 2	13	0
13 ÷ 2	6	1
6 ÷ 2	3	0
3 ÷ 2	1	1
1 ÷ 2	0	1

Read upwards: **110101**.

Binary addition and subtraction

ADDITION RULES

0 + 0 = 0, 0 + 1 = 1, 1 + 0 = 1

1 + 1 = 10 (write **0**, carry **1**)

1 + 1 + 1 = 11 (write **1**, carry **1**)

Example: 11011 + 01101

```
carry   1 1 1 1 1
          1 1 0 1 1
      +   0 1 1 0 1
        -----------
        1 0 1 0 0 0
```

- Column by column from the right: 1+1 = 0 carry 1; 1+0+1 = 0 carry 1; 0+1+1 = 0 carry 1; 1+1+1 = 1 carry 1; 1+0+1 = 0 carry 1; then the final carry 1.

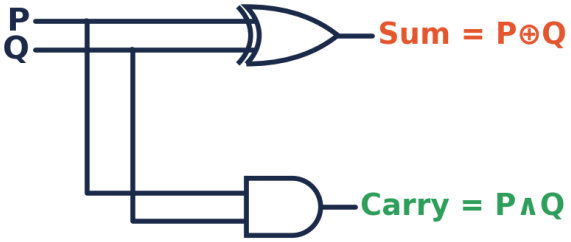
dawaralibhatti@gmail.com

- Result **101000**. Check: 27 + 13 = 40 = 32 + 8. **Correct.**

Subtraction is done by adding the **two's complement** of the number being subtracted (below).

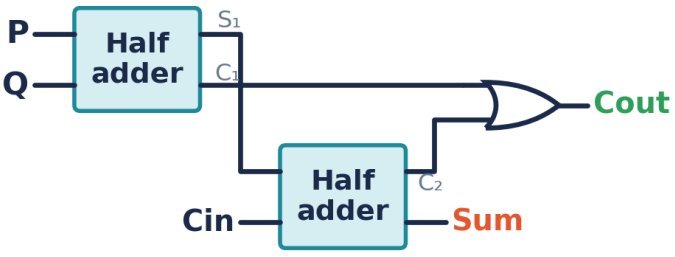
Adder circuits

HALF-ADDER



Adds two bits. No carry-in.

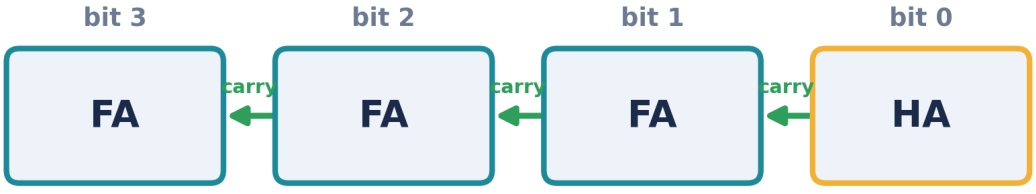
FULL-ADDER



Sum = P⊕Q⊕Cin
Cout = (P∧Q) ∨ ((P⊕Q)∧Cin)

Circuit	Inputs	Sum output	Carry output
Half-adder	P, Q	**P⊕Q**	**P∧Q**
Full-adder	P, Q, carry-in C	**P⊕Q⊕C**	**(P∧Q) ∨ ((P⊕Q)∧C)**

A **full-adder is two half-adders plus an OR gate**. Chaining n full-adders adds n-bit numbers: the carry-out of each stage feeds the carry-in of the next. The half-adder is used for the rightmost bit since there is no carry-in.



The carry-out of each stage feeds the carry-in of the stage to its left. The rightmost bit uses a half-adder (no carry-in).

dawaralibhatti@gmail.com

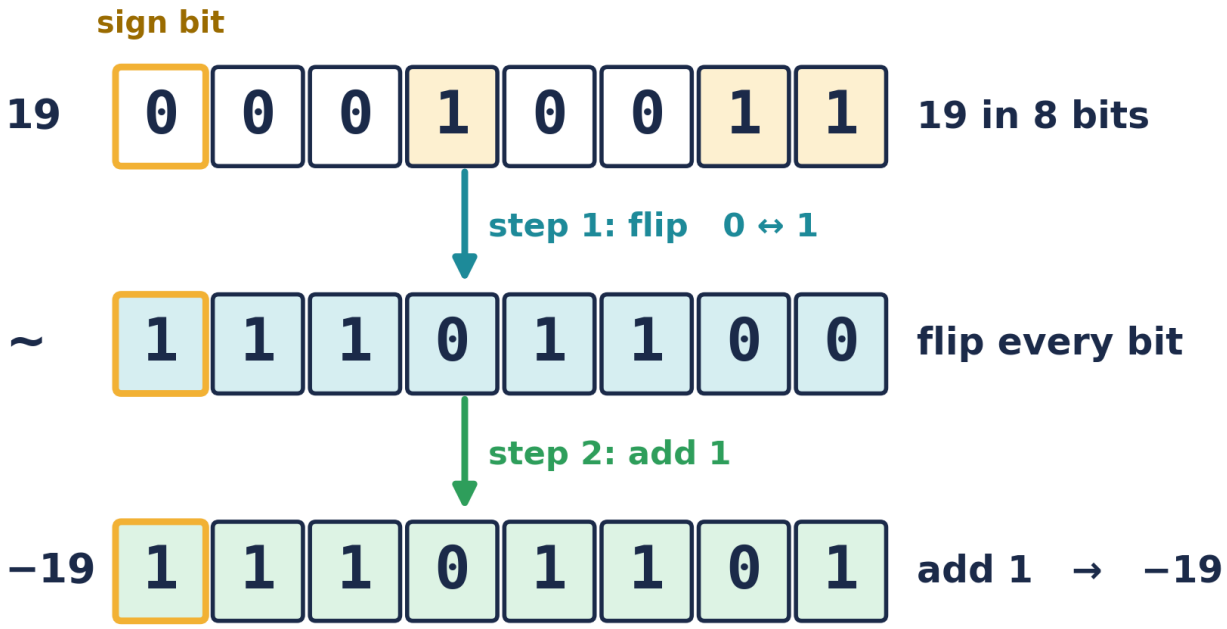
Two’s complement and negative integers (8-bit)

THE BASICS

In 8 bits the leftmost bit is the **sign bit**: **0** means non-negative, **1** means negative.
The range is **−128 to +127**.

To write a negative number −a in 8 bits

1. Write **a** in 8 bits.
2. **Flip** every bit (0 to 1, 1 to 0).
3. **Add 1**.



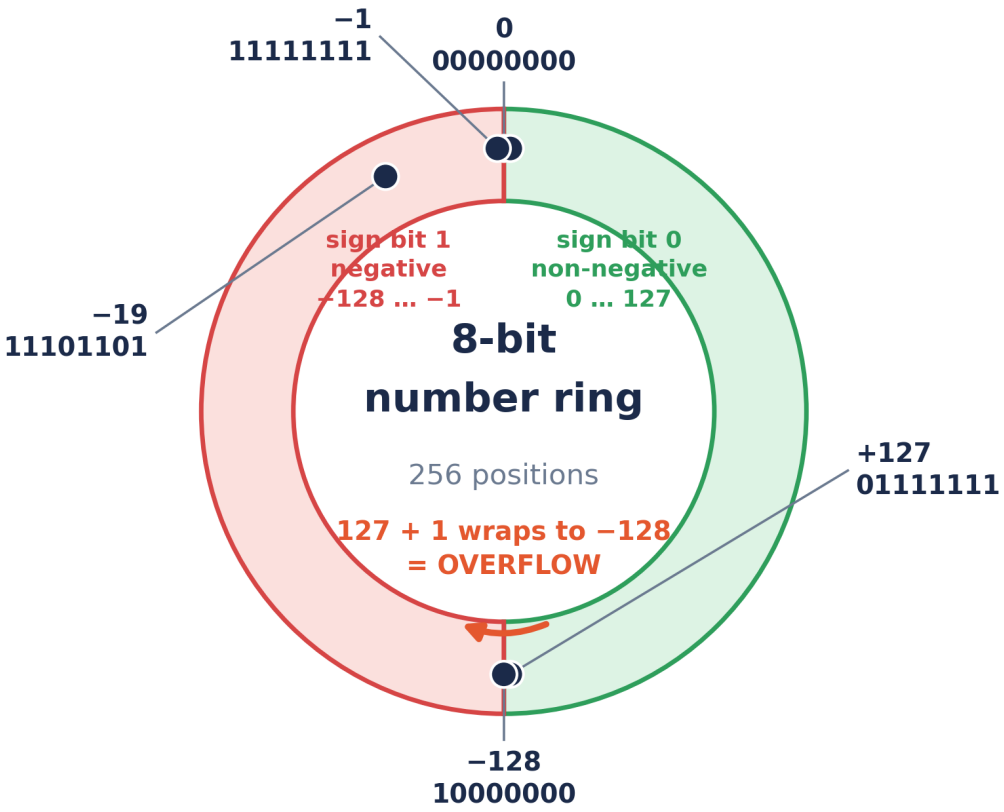
−19 in 8-bit two’s complement.

Check: 256 − 19 = 237 = 11101101.

dawaralibhatti@gmail.com

To read an 8-bit number that starts with 1

Flip the bits, add 1, and the result is the magnitude; the number is negative. 11101101 flips to 00010010, plus 1 = 00010011 = 19, so the number is **−19**.



Think of the 256 eight-bit patterns as a ring. Going past +127 wraps round to −128.

Computer addition with negatives

Add the 8-bit forms normally and **discard any carry out of the 8th bit**.

Example: **19 + (−7)**. −7 = 11111001 (00000111, flip to 11111000, add 1).

dawaralibhatti@gmail.com

```
  00010011    (19)
+ 11111001    (−7)
-----------
1 00001100
^ drop this carry
  00001100  = 12
```

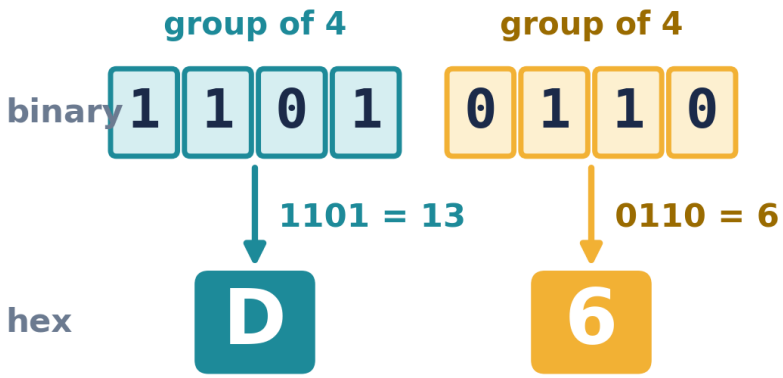
WATCH OUT

Overflow: if the true answer is outside −128 to 127, the 8-bit result is wrong. Two positives that add to a result starting with 1 signals overflow.

Hexadecimal

- Base 16. Digits 0–9 then **A = 10, B = 11, C = 12, D = 13, E = 14, F = 15**.
- **Binary → hex:** split into groups of 4 bits from the right and convert each group. 11010110 = 1101 | 0110 = **D6**.
- **Hex → binary:** expand each digit to 4 bits. 3F = 0011 1111.
- **Hex → decimal:** D6 = 13×16 + 6 = **214**.

dawaralibhatti@gmail.com



D6 (hex) = 13 × 16 + 6 = 214 (decimal)

Group values to memorise

Binary	Hex	Binary	Hex
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

dawaralibhatti@gmail.com

2.1 Predicates and quantified statements I

Predicates and domains

- A **predicate** is a sentence with variables that becomes a statement when the variables are replaced by values. P(x): “x is even.”
- The **domain D** of a variable is the set of values it can take, for example $\mathbb{R}$, $\mathbb{Z}$ or $\mathbb{Z}^+$.
- The **truth set** of P(x) is the set of all x in D that make P(x) true.
- Variables, constants and expressions (Lecture 1): in **2x + y = 0**, 2 and 0 are constants, x and y are variables, and the whole equation is an expression.

Symbol	Meaning
$\mathbb{Z}$	integers
$\mathbb{Q}$	rationals, p/q with q ≠ 0
$\mathbb{R}$	reals

The universal quantifier ∀

∀ FOR ALL
“∀x ∈ D, P(x)” means P(x) is true for **every** x in D.
Read “for all”, “for every”, “for each”, “for any”.

- **True** if P(x) holds for every element of D (you need a general proof, or checking every element for a small domain).

dawaralibhatti@gmail.com

- **False** if there is even **one counterexample**, an x in D where P(x) is false.

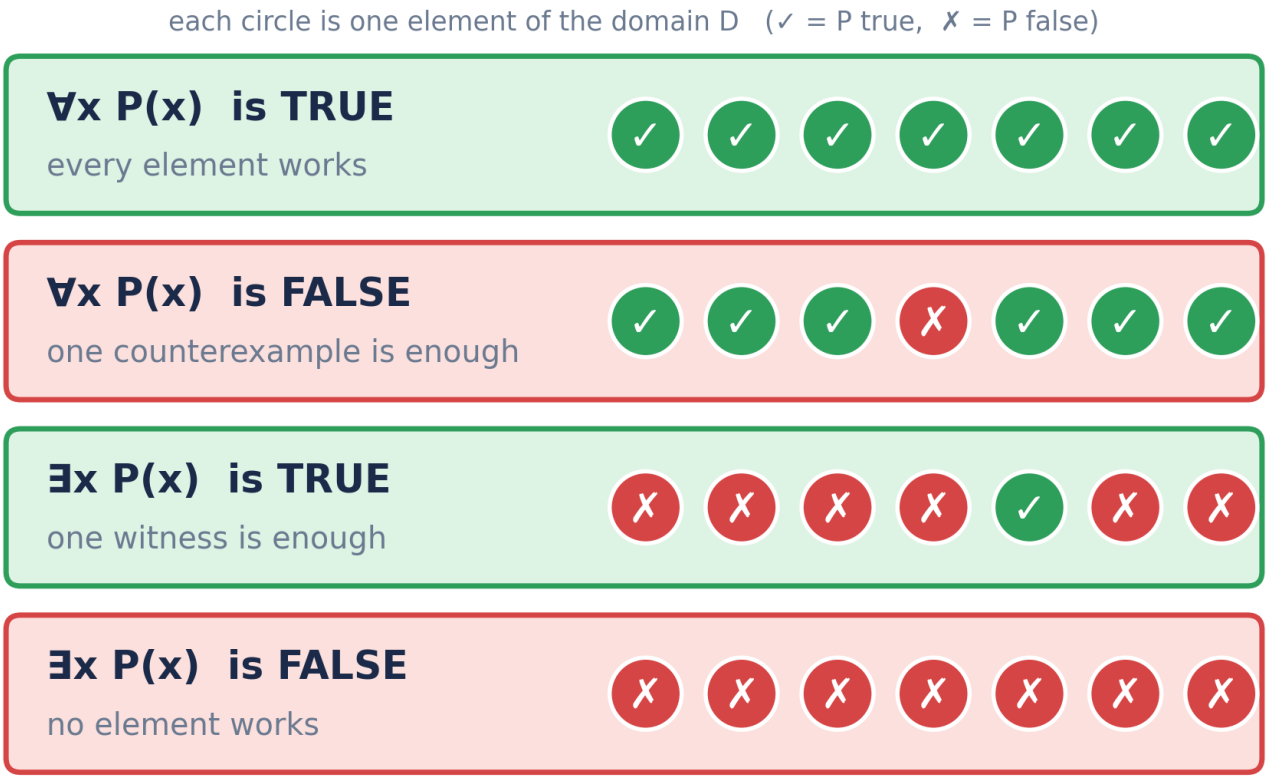
Example: $\forall x \in \mathbb{R}, x^2 \geq 0$ is **true**. $\forall x \in \mathbb{R}, x^2 > 0$ is **false**: the counterexample is $x = 0$.

The existential quantifier ∃

> **∃ THERE EXISTS**
> **"∃x ∈ D such that P(x)"** means there is **at least one** x in D for which P(x) is true.
> Read "there exists", "there is", "for some", "for at least one".

- **True** if you can find **one example** (a witness).
- **False** only if P(x) is false for **every** x in D.

Example: $\exists n \in \mathbb{Z}^+$ such that $n^2 = n$ is **true** ($n = 1$).



How to decide truth values: ∀ needs every element to work, ∃ needs just one.

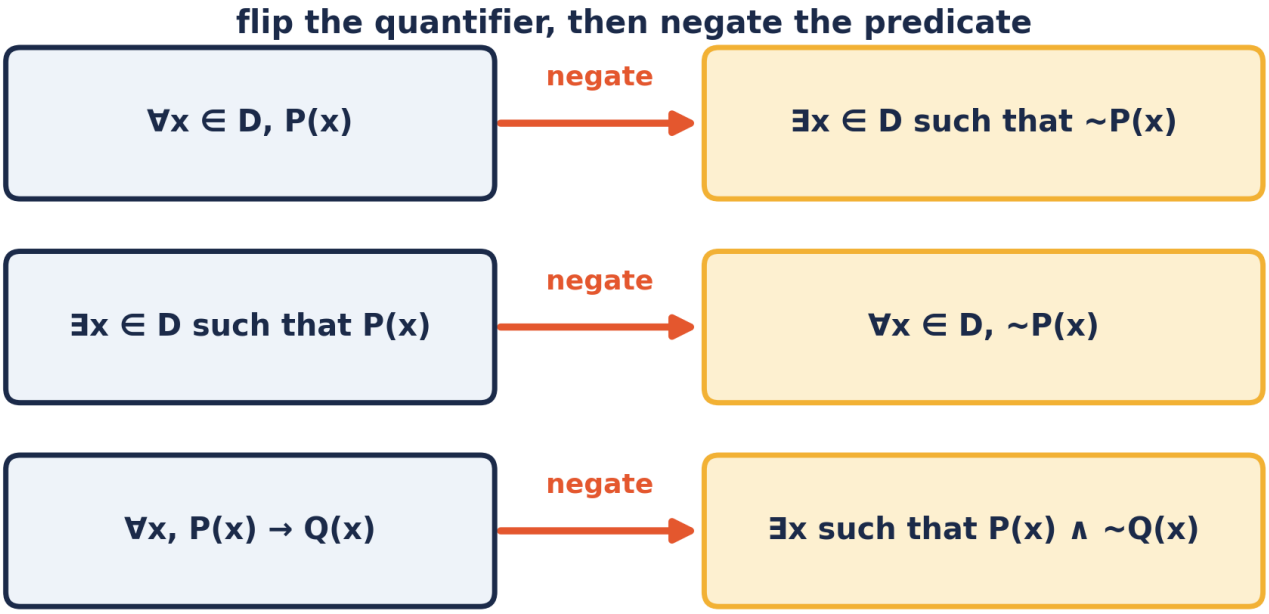
dawaralibhatti@gmail.com

Universal conditional statements

Most mathematical claims are of the form **∀x, P(x) → Q(x)**. Example: “If a real number is greater than 2, then its square is greater than 4.” means ∀x ∈ ℝ, x > 2 → x² > 4.

- It is **false** if there is an x with P(x) true and Q(x) false (a counterexample).
- It is **vacuously true** for x where P(x) is false. Those x never count against the statement.
- Existential statements are usually **∃x such that P(x) ∧ Q(x)**, with **AND**, not with →.

Negating quantified statements



THE RULE

Flip the quantifier, then negate the predicate.

The last row combines this with ~(p→q) ≡ p∧~q from section 1.2.

dawaralibhatti@gmail.com

English statement	English negation
All mathematicians wear glasses.	Some mathematician does not wear glasses.
Some dogs have wings.	No dog has wings (every dog does not have wings).
Every student passed the quiz.	At least one student did not pass the quiz.
Every even number is divisible by 4.	There is an even number that is not divisible by 4.
If x is prime then x is odd. (for all x)	There is a prime x that is not odd (x = 2).

WATCH OUT

The negation of “all are” is **NOT** “all are not”. It is **“at least one is not”**.

Equivalent forms

- “All squares are rectangles”, “Every square is a rectangle”, “For all x, if x is a square then x is a rectangle” and “If x is a square then x is a rectangle (for every x)” all mean **∀x, Square(x) → Rectangle(x)**.
- “**Some**” means ∃, and “**no**” means ∀ with a negation: “No dogs have wings” is **∀x, Dog(x) → ~Wings(x)**.
- “∀x ∈ D, P(x)” and “∀x, x ∈ D → P(x)” say the same thing.

dawaralibhatti@gmail.com

Bound variables, scope, implicit quantification

- In ∀x P(x), the variable x is **bound** by the quantifier. The **scope** is the part of the statement the quantifier governs. Renaming a bound variable does not change meaning: **∀x P(x) ≡ ∀y P(y)**.
- A variable not covered by any quantifier is **free**. A statement with a free variable is a **predicate**, not a statement.
- **Implicit quantification:** "$x^2 \geq 0$" in a theorem or definition is understood as "for all x". Similarly, "If x is a square then x is a rectangle" is understood as universal.

Tarski's world

A grid of coloured shapes (triangles, squares, circles) used to practise quantified statements. To judge a statement:

- for **∀** look for a single **counterexample**;
- for **∃** look for a single **witness**.

> **EXAM TIP**
> Always check the **domain** (all objects, or only the triangles, for example) before deciding.

dawaralibhatti@gmail.com

Quick revision

Ten things to write from memory

1. Truth tables of ~, ∧, ∨, ⊕, →, ↔, ↑, ↓.
2. De Morgan: **~(p∧q) ≡ ~p∨~q** and **~(p∨q) ≡ ~p∧~q**.
3. Implication law: **p→q ≡ ~p∨q**.
4. Negation of a conditional: **~(p→q) ≡ p∧~q**.
5. Contrapositive: **p→q ≡ ~q→~p**. Converse and inverse are **not** equivalent to p→q.
6. p **only if** q = p→q. **Necessary:** s→r. **Sufficient:** r→s.
7. **p↔q ≡ (p→q)∧(q→p)** and **p⊕q ≡ ~(p↔q)**.
8. **Valid argument:** every critical row has a true conclusion.
9. **Fallacies:** converse error and inverse error.
10. Negation of quantifiers: **flip ∀ and ∃, then negate the predicate**.

Common mistakes

- ✗ Writing ~(p∧q) as ~p∧~q.
- ✗ Treating the converse as equivalent to the original.
- ✗ Making p→q false when p is false. It is **true**.
- ✗ Forgetting that “p only if q” is p→q, not q→p.
- ✗ Negating “all” as “none”.
- ✗ Skipping the law name on a simplification step.
- ✗ Wrong number of rows: n variables need **2^n** rows.

dawaralibhatti@gmail.com

Self-test

1. Build the truth table for **(p∨q)∧~(p∧q)**. What connective is it?
2. Show that **~(p∨(~p∧q)) ≡ ~p∧~q** using the laws.
3. Write the converse, inverse and contrapositive of “If it rains, the match is cancelled.”
4. Negate: “If Sara studies, she passes.”
5. Is **p, p→q ⊢ q** valid? Is **q, p→q ⊢ p** valid?
6. Express **~P** using only NAND gates.
7. Write **−23** in 8-bit two’s complement.
8. Negate: “Every student in the class has a laptop.”

dawaralibhatti@gmail.com

Self-test answers

Try the questions first. No peeking!

1. The table is **F, T, T, F** for the rows TT, TF, FT, FF. It is **XOR (p⊕q)**.
2. ~(p∨(~p∧q))
   ≡ ~p ∧ ~(~p∧q) *(De Morgan)*
   ≡ ~p ∧ (p∨~q) *(De Morgan, double negation)*
   ≡ (~p∧p) ∨ (~p∧~q) *(distributive)*
   ≡ c ∨ (~p∧~q) *(negation)*
   ≡ ~p∧~q *(identity)*
3. **Converse:** If the match is cancelled, it rains.
   Inverse: If it does not rain, the match is not cancelled.
   Contrapositive: If the match is not cancelled, it does not rain.
4. Sara studies **and** she does not pass.
5. The first is **valid** (modus ponens). The second is **invalid** (converse error).
6. **P ↑ P**.
7. 23 = 00010111, flip to 11101000, add 1 to get **11101001**.
8. There is a student in the class who does **not** have a laptop.